Accessibility

Importance of Accessibility and Good HTML Structure

The importance of accessibility and good HTML structure cannot be overstated in modern web development. Accessibility ensures that everyone, regardless of their abilities or the assistive technologies they use, can access and interact with web content.

What is accessibility?

Accessibility involves creating products and services that everyone can use. In the context of web development, it's making websites that everyone can understand and interact with, including people with **visual, auditory, motor**, and **cognitive** disabilities.

Some examples of disabilities that can impact users' online experience include:

- Blindness.
- Low vision.
- Color blindness.
- Deafness.
- Difficulty using keyboards, mice, or touchscreens.
- Attention disorders
- Memory issues.
- Difficulty speaking or understanding spoken language.
- Sensitivity to flashing lights.

These are just a few of many conditions that can impact users around the world.

To help you create accessible websites, the World Wide Web Consortium, known as W3C, developed a set of international standards that you can follow to make your websites more accessible and easier to use for people with disabilities.

These standards are known as "The Web Content Accessibility Guidelines" (WCAG).

These guidelines are designed with four core principles in mind, known as **POUR**.

- `P` stands for **Perceivable**. Users must be able to perceive the information that you are presenting. For example, you can provide alternative text for images, so users who access your website with a screen reader can understand them.
- `O` stands for **Operable**. Users must be able to interact with the user interface. For example, you can make sure that all functionality is accessible through the keyboard too, not just the mouse.
- `U` stands for **Understandable**. Users must be able to understand the information. For example, you can avoid complex sentences and use simple language as much as possible.
- `R` stands for **Robust**. A wide range of browsers and other tools, including assistive technologies, must be able to interpret the content.

Using semantic HTML is very helpful for making your website compatible with different browsers and **assistive technologies**.

If your content doesn't follow any one of these core principles, not everyone will be able to use your website.

To check if you are following these guidelines correctly, you can access the Quick Reference of the **World Wide Web Consortium**. There, you will find a comprehensive list of criteria and techniques.

Accessibility is essential for web development. By developing with inclusivity in mind, you can ensure that everyone can access and engage with your content, promote equality and create a better user experience for everyone around the world

---

What are screen readers and who uses them?

Screen readers are **assistive technology** programs that help blind and visually impaired people use computers and mobile devices.

Screen readers are not just tools for the blind and visually impaired to access computers and mobile devices.

They empower these individuals to access education, work opportunities, and social media. This ensures digital inclusion and enhances their ability to participate fully in society.

There's a common misconception that screen readers are text-to-speech devices.

However, **text-to-speech** is just one of the features of a screen reader. Some screen readers even render the text to **braille** output instead of speech.

Apart from text-to-speech and braille output, other notable features of screen readers are navigation aids and web browsing assistance.

Screen reader programs are also not only made for the blind and visually impaired. Dyslexic individuals and people with cognitive disabilities also use screen readers. All the popular operating systems out there have screen readers built into them.

macOS and **iOS** both have **VoiceOver** built-in. You can turn it on on your computer by pressing `CMD + F5`. You can access it on iPhones through Settings.

Windows computers have **Narrator** built-in. You can turn it on by pressing `WIN + CTRL + ENTER`. NonVisual Desktop Access (**NVDA**) and Job Access With Speech (**JAWS**) are also available for Windows computers. NVDA is free and open-source, while JAWS is paid.

Linux has ***Orca*** for the desktop environment and ***Speakup*** for the Linux terminal.

Android phones have **TalkBack** built-in. You can turn it on by accessing *Settings > Special Function > Accessibility > Talkback*.

Some Android devices also have **Ella** and Select to Speak built in.

One major challenge for screen reader users is that many software developers don't design their products with screen-reader friendliness and accessibility in mind.

Even though accessibility is a broad topic, every developer needs to learn how to make their web software accessible for the blind and visually impaired, as well as other groups of people with disabilities.

This demonstrates empathy and a commitment to inclusivity, ensuring that all users can benefit from their work.

What are Large Text or Braille Keyboards, and Who Uses Them?

Large text and braille keyboards are designed for users with visual disabilities. In Large Text Keyboards, also called Large Print Keyboards, the letters, numbers, and symbols are larger compared to standard keyboards. This design is helpful for people who may find smaller text in the keys difficult to see. Most of them also have enhanced contrast and brightness.

A large print keyboard made by the brand MaxiAids has yellow keys with black, big, and bold letters, numbers, and symbols on them. This is helpful for people with low vision.

Another keyboard is the black large print keyboard with white print on the keys. This keyboard is also backlit, so users can adjust its brightness to different lighting conditions.

While large print keyboards provide visual cues for users with low vision, braille keyboards provide a completely tactile experience for people with more severe vision disabilities, including people who are blind.

Braille is a tactile reading and writing system. It consists of raised dots arranged in specific patterns to represent letters, numbers, and punctuation.

Braille keyboards use this system to help users find the right keys on the keyboard by feeling these patterns with their fingers. The keys have raised dots in patterns that represent letters, numbers and symbols.

And some keyboards combine both approaches- large fonts and braille patterns in the keys. This is helpful for people with visual disabilities and for people who are learning braille.

Large text and braille keyboards are tools that empower people with visual disabilities. By providing alternative input methods, these assistive technologies ensure that everyone can be part of the digital world.

What are alternative pointing devices such as trackballs, joysticks, and touchpads used for?

Alternative pointing devices are input devices that make good alternatives to the traditional mouse. They are essential for improving computer accessibility for those with disabilities, forelimb impairments, and limited mobility.

Common examples of alternative pointing devices are trackballs, joysticks, and touchpads.

A trackball is a stationary pointing device that consists of a large, movable ball within a socket. It also includes additional buttons for clicking and performing other functions.

Unlike the traditional mouse, which requires movement around a surface to control the cursor, a trackball remains in place. Users manipulate the ball directly with their fingers, thumb, or palm to move the cursor on the screen.

Some traditional mice also have a trackball on the top or side. These mice could make a good starting point if you want to gradually switch to a trackball.

Trackballs reduce the physical movement the user needs for navigation, making them ideal for users with mobility issues. Apart from that, if you need high precision and have limited desk space, a trackball is more ideal than a traditional mouse.

A joystick is a pointing device primarily designed for games and certain industrial applications like machinery control. It consists of a lever that pivots up, down, left, and right, and often includes additional buttons for various actions

Joysticks provide precise control over movement and actions within digital environments. This makes them popular for flight simulators, cranes, driving games, and other applications requiring precise directional input.

Because joysticks accommodate larger and more deliberate movements, they are beneficial for individuals with tremors and unsteady hands.

They also reduce the strain and pain that comes with repetitive movements, making them ideal for individuals with arthritis and carpal tunnel syndrome.

A touchpad is a flat, touch-sensitive device built into laptops and some keyboards. It allows users to control the cursor on the screen by sliding their fingers across its surface.

Apart from the surface for cursor control, touchpads also feature buttons that emulate the actions of a traditional mouse, such as right-click and left-click.

Most people see touchpads as a better alternative to a mouse because they significantly enhance navigation by supporting multi-touch gestures like pinch-to-zoom, two-finger scrolling, tap-to-click, and three-finger swipes.

Touchpad is ideal for individuals with low arm or hand movement because the forelimb is almost always stationary while using it. It is also suitable for people with arthritis and joint pain because they don't get to move their arms too much

What are screen magnifiers used for?

Screen magnifiers are tools that help people with low vision and other visual impairments better access digital content and the web.

Let's delve deeper into what these tools are and the role they play in digital content accessibility.

Screen magnifiers work by enlarging texts, graphics, and other elements on a computer or mobile device screen. Many screen magnifiers allow users to enlarge the display by more than 200%. Users can then navigate the page using their pointing device or keyboard. Additionally, most magnifiers offer customizable zoom percentages and other features in their settings.

Screen magnifiers primarily help people with low vision read text, as small fonts in documents or applications can be challenging for them. By enlarging the text, they can read emails, articles, and other content without straining their eyes. Screen magnifiers also assist with web browsing. They help users locate and click on buttons, links, and other interactive elements that might be difficult to see. This improved visibility ensures that users can browse websites, fill out forms, and engage in online activities without difficulty.

Therefore, software developers need to make their digital products accessible to people with low vision. Some considerations include:

- Using scalable fonts so the user can resize the page without the layout breaking.
- Ensuring the user interface adapts to different screen sizes through responsive design.
- Using high-contrast color schemes and customizable colors.
- Implementing a non-sticky and tiny navbar so users can still see content when using magnifiers.
- Using regular HTML text instead of images of text.
- Providing feedback directly next to the element that triggers it, and more.

All the mainstay Operating Systems have at least one magnifier built into them by their manufacturers:

- macOS and iOS both have Zoom. You can turn it on on Mac by going to Settings, filter by Accessibility, and then click on "Zoom". Toggle the "Use keyboard shortcuts to zoom" option to enable it.
    - You can turn it on on iPhone through *Settings > Accessibility > Zoom*.
- Android devices have Magnification. To turn it on, go to *Settings > Special Function > Accessibility> Magnification*. Since this may vary from device to device, you can search for "Magnification" on the settings homepage to access it.
- Windows has Magnifier. You can use it by going to *Settings > Ease of Access > Magnifier*.
- The magnifiers for Linux operating systems vary. It is either Zoom or Magnifier.

Apart from the ones built into operating systems, some useful third party screen magnifiers are:

- ZoomText for Windows.
- ClaroView for both Mac and Windows.
- iZoom for Windows.
- Zoomify - Screen Magnifier for Mac.
- LunarPluse for Windows.
- Loupe for Mac

What is voice recognition software used for?

Voice recognition software helps people with disabilities interact with computers and other digital devices. Let's discuss what voice recognition software is and the role it plays in digital inclusion.

In the context of accessibility, voice recognition tools let people with disabilities use their voice to pass commands to perform various tasks instead of using traditional input devices like keyboards and mice. This includes writing emails and other documents, surfing the net, and controlling smart home devices.

Because voice recognition software tools eliminate the need for physical interaction, they empower people with disabilities with significant independence and control over their environment.

Here are the specific people who may find voice recognition software significantly helpful:

- People with visual impairments, including those with low-vision or blindness.
- Individuals with mobility impairments, such as limited use of hands and arms or conditions like arthritis and carpal tunnel syndrome.
- Those recovering from hand or arm injuries.
- Individuals with cognitive disorders, like memory issues or attention deficit disorders.
- Elderly individuals who might find it easier to use voice commands.

Note that people with disabilities are not the only ones who use voice recognition technology. Law enforcement agencies, gamers, drivers, and busy professionals also use voice recognition tools.

A few examples of voice recognition software that allows people to interact with their computer include Voice Control for macOS/iOS, Voice Access for Android, and Windows Speech Recognition for Windows (referred to as Voice Access in the most recent versions of Windows). Dragon by Nuance is a popular third-party voice recognition software for Windows

---

What are some common accessibility auditing tools to use?

Accessibility is a crucial yet often overlooked aspect of digital content. When making your digital content accessible, it's important to ensure it meets accessibility standards.

An accessibility auditing tool is an application that helps you improve the accessibility of your digital content by reporting accessibility issues that can be easily found through automated testing. This content includes websites, web applications, and mobile apps.

It is important to note that while automated accessibility tools have a role in improving accessibility, they typically will only find about a third of all possible

accessibility issues. Therefore, it is important not to rely on them entirely to evaluate the accessibility of your content. Manual testing, preferably by people with disabilities, will always be required to ensure that your content is as accessible as possible.

Let's look at some free tools that can help you improve the accessibility of your digital content.

Google Lighthouse is a popular web metric checker you can use directly within Chrome DevTools or online. This means you can check not only live websites but also locally-developed ones.

The metrics you can check include accessibility, SEO, best practices, and performance.

To use Lighthouse, open your DevTools by pressing `F12` and switching to the Lighthouse tab. Select the metrics you want to check, choose the device you want to test on, and click the "Analyze page load" button.

An accessibility score will appear after the check is complete, along with a list of any issues that need fixing.

If you want more reliable metrics, consider using the web version. The downside is that it doesn't support testing local websites. You can access the web version on `pagespeed.web.dev`.

WAVE is another reliable accessibility checker you can use as a Chrome extension or on the web. All you need to do is enter the URL of your website and a comprehensive accessibility report will be generated for you. This report includes accessibility features implemented, ARIA, and contrasts.

The IBM Equal Accessibility Checker is another robust tool for improving digital content accessibility. With it, you can scan your websites for accessibility issues and generate a detailed report.

You can use it as a Chrome extension or Firefox add-on.

To use the IBM Accessibility Checker as a Chrome extension, download it from the Chrome web store. Open your Devtools by pressing `F12` and selecting the "Accessibility Checker" tab located in the Elements panel. Click the scan button to start the check and a report will be generated for you. You can export the report as a spreadsheet and an HTML file by clicking the "Export XLS" button.

Please keep in mind, while these automated tools help you make your content more accessible, a perfect score from any of them does not mean that your content is entirely accessible. The range of issues that these tools test for is

limited, and manual testing will always be needed to ensure a more accessible experience for everyone

---

Working With Accessible Tables and Forms

Accessible tables and forms are crucial for creating inclusive web content. When building tables, always use proper `<thead>`, `<tbody>`, and `<tfoot>` tags, along with `<th>` for headers, and `scope` attributes to clearly define relationships between cells.

How does proper heading level structure affect accessibility?

In an earlier lecture video, you learned about proper heading level structure. In this video, you will learn how a good heading structure affects accessibility.

Proper use of headings creates a visual hierarchy for users to navigate and understand your web page. Using a logical heading hierarchy allows screen reader users to understand the structure of your content and navigate your content quickly. Creating appropriate heading text that accurately describes the content that follows helps everyone find the information they need on your site. As an additional benefit, well-formed headings may also improve the SEO of your site.

Think of headings as the foundation of your site. Without a good foundation, the accessibility of your content will suffer.

Let's look at how you can make your web projects friendly for people who use assistive technologies by using headings correctly.

Headings, ranging from `h1` to `h6`, create a navigational structure for screen reader users. Screen readers can list all headings on a page, allowing users to jump directly to the sections they need. So, correctly arranging headings is important for helping these users avoid unnecessary content and finding the information they are looking for quickly.

People with partial sight or cognitive disabilities need to process information quickly and easily to reduce cognitive load.

While proper arrangement of headings is important, it is also necessary to make heading texts clear and descriptive.

Here are some key practices to follow to use headings properly:

- Use headings in a hierarchy that reflects clear organization. For example, the page title should be an `h1`, major sections should be introduced with `h2` headings, subsections with `h3` , and so on, down to `h6`.
- Don't skip from `h1` to `h3`, or from `h2` to `h4`, and so on.
- Use clear and descriptive text that summarizes the content that follows each heading.
- Don't use a heading in isolation – some content must follow the heading.
- Use appropriate headings where necessary instead of formatting text to look like headings.
- Each page should have a single `h1` element representing the main topic or title.

Here's a basic markup that represents how you should use headings on a page:

```
<!-- Page title -->
<h1>What is HTML</h1>

<!-- First section -->
<section>
  <h2>Introduction to HTML</h2>
  <p>
    HTML stands for HyperText Markup Language. It is the standard language f
    creating web pages.
  </p>
</section>
<!-- Second section -->
<section>
  <h2>History of HTML</h2>
  <p>HTML began to take shape in the early 90s</p>
  <h3>Origins</h3>
  <p>
    HTML was created by Tim Berners-Lee in 1991. It has evolved significantly
    over the years.
```

```
  </p>
</section>
```

What are best practices for tables and accessibility?

When we see a table, we immediately start making visual associations between the data and the headers.

For example, let's say we have a table representing our pets. We have two dogs and two cats and the table is showing us their names and ages. While a sighted person may be able to understand the relationships in this table, making the connections between the values and the headers is much harder for people who use screen readers to navigate the table.

As a web developer, you are in charge of creating these associations and structuring your HTML markup in a way that is also easy for screen reader users to interpret.

So, let's see how you can create accessible tables that everyone can understand. The first best practice that we will cover is using table caption. With the `caption` element, you can write the caption (or title) of a table, so users, especially those who use assistive technologies, can quickly understand the table's purpose and content. You should place the `caption` element immediately after the opening tag of the `table` element. This way, screen readers and other assistive technologies can provide more context by announcing the caption before reading the content.

```
<table><caption>Our Pets</caption><!-- Table Rows and Columns -->
</table>
```

Now let's talk about row and column headers. Headers are special cells, typically found at the start of a row or column, that describe the type of data stored in the row or column. You can define a row or column header with the table header element, `th`.

For example, the code below creates a table for two pets. Every row has a row header (the name of the pet) and every column has a column header, which describes what the data in the column represents (age and type).

```
<table><caption>Our Pets</caption><thead><tr><!-- Column Headers -->
    <th>Name</th><th>Age</th><th>Type</th></tr></thead><tbody><tr
><th>Nora</th> <!-- Row Header -->
    <td>5</td><td>Dog</td></tr><tr><th>Gino</th> <!-- Row Header -->
    <td>2</td><td>Cat</td></tr></tbody></table>
```

Notice that the above code has a `caption` element immediately after the opening `table` element. Then, inside the table head (`thead`) element, it has the column headers (`Name`, `Age`, and `Type`). In the second and third rows, inside the table body (`tbody`) element, we find the data for each one of our pets. The names of the pets are the row headers because they are inside table header elements (`th`).

Associating the data cells with their corresponding headers is also very important for screen readers. The `scope` attribute determines if a header is a row header or a column header. Screen readers may guess this correctly from the table's structure, but it's usually recommended to explicitly indicate the `scope` to ensure clarity.

The `scope` attribute has four possible values. The two you will use most often are `col` for column and `row` for row. In the code below, you can see that we added the `scope` attribute to the column and row headers. The three column headers (`Name`, `Age`, and `Type`) have a `scope` of `col`, column.

The two row headers (`Nora` and `Gino`) have a `scope` of `row`.

```
<table><caption>Our Pets</caption><thead><tr><!-- Now they have scop
e -->
    <th scope="col">Name</th><th scope="col">Age</th><th scope="co
l">Type</th></tr></thead><tbody><tr><th scope="row">Nora</th><td>5
</td><td>Dog</td></tr><tr><th scope="row">Gino</th><td>2</td><td>
Cat</td></tr></tbody></table>
```

If a column or row header spans across multiple cells, the `scope` will also be applied to each one of the cells individually. Here's an example of that:

	Name	**Age**
Dogs	**Nora**	5
	Gino	2

Cats	**Lulu**	10
	Elizabeth	6

In this table, the cell with `Nora`'s age (`5`) will have one column header (`Age`) and two row headers (`Dogs` and `Nora`). `Gino`'s age (`2`) will also have one column header (`Age`) and two row headers (`Dogs` and `Gino`).

However, some screen readers may not be able to interpret tables with complex structures, so you should also try to flatten the table as much as possible to avoid row and column headers that span across multiple cells.

Your goal should always be to make sure users can access this information, even if their screen readers can handle complex table structures.

Now, for cell width, it's recommended to avoid using fixed values. You should use relative values instead, like percentages. Also, try to avoid defining cell height. This will allow users to adjust the text size to fit their needs.

And finally, you should let the browser determine the table width whenever possible, to reduce the need for horizontal scrolling.

HTML tables are essential for presenting structured data in an accessible and understandable format. By following these accessibility guidelines, you can create tables that are easy to understand for everyone.

Why is it important for inputs to have an associated label?

Labels on input fields might seem like small details, but they play a crucial role in making forms easier for everyone to use.

Labels help people with disabilities, make forms more user-friendly for everyone, and ultimately improve form conversion rates.

Let's dive into why adding labels to input elements is important for accessibility and usability.

Screen readers often rely on labels to describe the purpose of input fields. In order for this to work properly, the label must be programmatically associated with the `input`. While there are several ways to do that, the most common is to use the HTML `label` element.

Here's how to associate a `label` with an `input`:

```
<form><label for="name">Your Name</label><input type="text" id="name" /></form>
```

In this example, the `for` attribute of the `label` element is associated with the `id` of the `input` element. This connection allows screen readers to announce the `label` when the `input` is in a focused state, allowing screen reader users to understand the purpose of the `input`.

Associating labels with all input fields also benefits SEO. With proper labels, search engines will understand the page content better, leading to a positive impact on the page search rankings.

But it is important to understand that you're making content for humans, not search engines. So, you should make sure the labels are as clear and descriptive as possible.

Introduction to ARIA

In these lecture notes, you will learn about working with ARIA roles.

What is the purpose of WAI-ARIA, and how does it work?

Making static content accessible can be relatively straightforward, but dynamic content can be more challenging. This is where WAI-ARIA comes in.

Let's look at what WAI-ARIA is, its purpose, how it works, and some examples.

WAI-ARIA stands for Web Accessibility Initiative - Accessible Rich Internet Applications. It's a specification that enhances accessibility for dynamic content and UI (User Interface) components.

Note that WCAG and WAI-ARIA are not the same. WCAG provides general guidelines for web accessibility, while WAI-ARIA offers specific rules for making dynamic and interactive content accessible for users of assistive technologies.

So, the primary purpose of WAI-ARIA is to improve accessibility for dynamic content and UI components that do not have native HTML equivalents.

WAI-ARIA works by introducing a set of attributes you can add to HTML elements to provide additional semantic information. These attributes are categorized into roles, states, and properties.

ARIA role defines the purpose of an element within a website or web app. Here is an example of setting the role to `button` for a `div` element.

```
<div role="button">Click Me</div>
```

By doing this you are indicating to assistive technology that the element is a button. Roles do not provide any functionality however. Merely giving this `div` a `role` of `button` will not make it act like a button. To make it look and behave like a button you would need to use CSS and JavaScript to get the desired result. It is always better to use the native `button` or `input` element with `type="button"` instead.

Since HTML alone does not provide a way to make custom UI components like tabbed navigation (tab list) accessible, ARIA roles can be very helpful.

Here's what a tablist could look like with the `role` attribute:

```
<div role="tablist" aria-label="Football Match Dashboard"><button
    role="tab"aria-selected="true"aria-controls="match-info-panel"id="mat
ch-info-tab">
      Match Info
  </button><button
    role="tab"aria-selected="false"aria-controls="player-stats-panel"id="pl
ayer-stats-tab">
    Player Stats
  </button></div>
```

Each tab in the tablist will have an associated tab panel that will display when its tab is selected. Each tab panel will have a `role` of `tabpanel`.

Here's what each of those tab panels could look like:

```
<div
  role="tabpanel"id="match-info-panel"aria-labelledby="match-info-tab">
<p>
    Details about the match, including date,
    time, venue, and current score.
  </p></div><div
  role="tabpanel"id="player-stats-panel"aria-labelledby="player-stats-ta
b"hidden
```

```
><p>
    Individual player statistics such as goals,
    assists, and minutes played.
  </p></div>
```

ARIA state describes the current status of an element, which can change based on user interactions. An example is the `aria-expanded` attribute for collapsible sections:

```
<button aria-expanded="false" aria-controls="more-info">
  More Info
</button><div id="more-info" hidden><p>This is additional information.</p
></div>
```

Again, this is merely describing the state, it does not provide any functionality. You would have to use JavaScript to change the state of the attribute when the user clicks the button.

ARIA properties provide additional details about elements. For example, the `aria-labelledby` property lets you connect an element to a specific label:

```
<h2 id="header-id">About freeCodeCamp</h2><button id="button-id" ari
a-labelledby="header-id button-id">Learn More</button>
```

This will make the elements understandable and navigable for users of assistive technologies.

To get the best out of WAI-ARIA, try to stick with native HTML whenever possible since it generally provides more accessibility out of the box.

Use WAI-ARIA only when HTML falls short, and don't forget to test with assistive technologies like screen readers, or have people with disabilities test your work. Also, make sure your WAI-ARIA states and properties update with the content in real time. Avoid overusing ARIA as it can be confusing many times.

What are ARIA roles?

ARIA stands for Accessible Rich Internet Applications.

ARIA roles specify the semantic meaning of HTML elements. They're essential for making web content accessible to people who use assistive technologies, like screen readers.

HTML has semantic and non-semantic elements, based on whether they convey meaning about their content.

Many semantic HTML elements already have an ARIA role assigned by default. For example, the `button` element has a default ARIA role of `button`.

But non-semantic elements don't have a role. For example, screen readers will not know how to interpret the purpose of a `div` if you don't specify its role explicitly.

To specify the ARIA role of an element, you just need to add the `role` attribute, like this `role="ARIA role"`, where value is the name of a role in the ARIA specification.

It is important to note that specifying a role on an element only does one thing: It informs assistive technology of the purpose of the element. It does not add any functionality or behavior to the element. If people expect a role to behave a certain way, it is up to you, the developer, to add that expected behavior. For example, adding a `role` of `button` to a `div` does not automatically make it clickable with a mouse or usable with a keyboard. It is the responsibility of the developer to add the expected behavior that allows the `div` to act like a button, and in most cases, it is just better to use the `button` element.

There are six main categories of ARIA roles:

- Document structure roles
- Widget roles
- Landmark roles
- Live region roles
- Window roles
- And Abstract roles

Let's take a look at them in more detail.

Document structure roles define the overall structure of the web page. With these roles, assistive technologies can understand the relationships between different sections and help users navigate the content.

However, most of the document structure roles are not used in modern web development because browsers already support equivalent semantic HTML elements, which should be prioritized whenever possible.

You should specify the roles that don't have an equivalent semantic element. For example: `toolbar`, `tooltip`, `feed`, `math`, `presentation`, `none`, and `note`.

There are other similar roles but these are the most commonly used ones. This is an example of a `div` with the `math` ARIA role. The `div` contains a mathematical equation.

```
<div role="math" aria-label="x squared + y squared = 3">
 x<sup>2</sup> + y<sup>2</sup> = 3
</div>
```

You'll also notice that the `div` has an `aria-label` attribute. The value of this attribute should be a string that represents the expression.

Widget roles define the purpose and functionality of interactive elements, like scrollbars.

Examples of widget roles include `scrollbar`, `searchbox`, `separator` (when focusable), `slider`, `spinbutton`, `switch`, `tab`, `tabpanel`, and `treeitem`.

Some of these roles have equivalent semantic elements. You should prioritize the semantic element over the role if there is one. For example, you should favor using the HTML `button` element over adding a `role` of `button` to a `div`.

Landmark roles classify and label the primary sections of a web page. Screen readers use them to provide convenient navigation to important sections of a page. You should use them sparingly to keep the overall layout simple and easy to understand. Examples of landmark roles are `banner`, `complementary`, `contentinfo`, `form`, `main`, `navigation`, `region`, and `search`. Each of these roles has a corresponding HTML equivalent, such as `header`, `footer`, `aside`, `form`, `main`, `nav`, `section`, and `search`. If you use the proper HTML elements to define the sections of your page then it is not necessary to explicitly add the `role` attribute to these elements.

Live region roles define elements with content that will change dynamically. This way, screen readers and other assistive technologies can announce changes to users with visual disabilities. These roles include: `alert`, `log`, `marquee`, `status`, and `timer`.

Window roles define sub-windows, like pop up modal dialogues. These roles include `alertdialog` and `dialog`. Please note that it is now considered a best practice to use the HTML `dialog` element and its associated JavaScript methods instead of manually creating a dialog.

And finally, we have abstract roles. These roles help organize the document. They're only meant to be used internally by the browser, not by developers, so you should know that they exist but you shouldn't use them on your websites or web applications.

With ARIA roles, you can create accessible and inclusive websites and web applications. They provide semantic information about the purpose and function of HTML elements.

Screen readers and other assistive technologies use this information to help users understand the content of your page and set expectations for how to use it, which helps ensure that everyone can have a great user experience.

What are the roles of the `aria-label` and `aria-labelledby` attributes?

It is important to make sure that all users, including those living with disabilities, can access websites without issues.

For people using screen readers, the `aria-label` and `aria-labelledby` attributes provide crucial information about page elements that might be unclear or invisible.

Let's look at what the `aria-label` and `aria-labelledby` attributes are and the roles they play in making the web accessible to people with visual impairments and related disabilities.

You'll notice both `aria-label` and `aria-labelledby` are prefixed with aria. So what does that mean? ARIA stands for Accessible Rich Internet Applications. It's a set of attributes prefixed with `aria-`, which lets developers communicate the purpose of elements to assistive technologies. The `aria-label` attribute is an invisible label for interactive elements. It adds a text label to an element that screen readers can read.

`aria-label` is especially useful for elements that do not have visible text but still need to be described by screen readers. For example, buttons with only icons often need `aria-label` to convey their purpose.

Here is an example:

```
<button aria-label="Search"><i class="fas fa-search"></i></button>
```

In this case, a screen reader may announce this button as `Search, button`, even though the button only contains an icon. The `aria-label` attribute tells screen readers what text to use in place of the icon.

If the button contained the text "Search" instead of an icon, then there would be no need for the `aria-label` attribute as the text would provide the label for the button.

For input elements, the `aria-label` attribute provides a label directly if there isn't a visible label associated with the input.

The `aria-labelledby` attribute does the exact same thing as the `aria-label` attribute, but instead of defining the text directly in the attribute, you use a reference to text that already exists on the page. The existing text must have an `id` attribute, which will be used for the reference value in the `aria-labelledby` attribute.

Here's an example:

```
<input type="text" aria-labelledby="search-btn"><button type="button" id
="search-btn">Search</button>
```

In this case, the text for the button is being used as the label for the search input. Screen readers will announce the input as something like `Search, edit`. If you later decide you want to change the button text to `Find`, the label for the input will automatically be updated to the new text since it is referencing the button text. Combining multiple `id` values into a single `aria-labelledby` attribute value is also possible. Here's how that works:

```
<div><span id="volume-label">Volume</span><span id="volume-detail
s">Adjust the volume level</span><input
  type="range"min="0"max="100"value="30"aria-labelledby="volume-lab
el volume-details"></div>
```

For the slider, the screen reader will look out for the content of the `volume-label` and `volume-details` elements and announce `Volume Adjust the volume level`.

You've seen that both `aria-label` and `aria-labelledby` attributes help screen readers understand what elements do. So, which one should you use? Since they both

provide the same functionality, either can be used, but there may be a few advantages to using `aria-labelledby` over `aria-label`:

- If someone is using a translation service to translate the content on your page, the text in an `aria-label` attribute may not always be translated.
- Using `aria-labelledby` can also help prevent a mismatch between the visible label text and the invisible label for screen reader users since updating the visible text will automatically update the invisible label.
- `aria-labelledby` can make it much easier to programmatically create complex invisible labels consisting of multiple sources of text.

One last note, do not use `aria-label` and `aria-labelledby` on an element at the same time. In this case, the invisible label for screen readers will always be determined by `aria-labelledby` and `aria-label` will be completely ignored.

What is the `aria-hidden` attribute, and how does it work?

If you ever need to display content while at the same time hiding it from people who use assistive technology, like screen readers, you can use the `aria-hidden` attribute.

You just need to add it to the HTML element that you want to hide and set its value to `true`, like you can see over here: `aria-hidden="true"`.

This attribute hides the element and all of its children from the accessibility tree, while keeping them visible on the page. Common use cases include:

- Icons and images that only have a decorative purpose.
- Duplicated content.

It is important to remember that `aria-hidden` only hides content from assistive technology, such as screen readers. If the content should be hidden from everyone, then you should not use `aria-hidden` to hide it. For example, a hamburger menu that is currently collapsed must be hidden from all keyboard users, not just screen reader users. In this case you could set the CSS `display` property to `none` on the menu to remove it from the DOM when it is collapsed.

You should never use `aria-hidden` to hide an element that is focusable with the keyboard. The `aria-hidden` attribute only removes the element from the

accessibility tree. It does not remove it from the DOM. Thus, it will still be possible for screen reader users to tab to the element, but their screen reader will not announce the element, effectively causing them to focus on "nothing".

Here's an example where we hide an icon from the accessibility tree by adding the `aria-hidden` attribute with a value of `true`.

We only keep the text exposed to assistive technologies to avoid any confusion that may arise from the redundancy of having both an icon and text for the same purpose.

```
<button><i class="fa-solid fa-gear" aria-hidden="true"></i><span class
="label">Settings</span></button>
```

You do not need to use `aria-hidden` when:

- The HTML element already has a `hidden` attribute.
- The element or the element's ancestor is already hidden with `display: none`.
- The element or the element's ancestor is already hidden with `visibility: hidden`.

In these three cases, the element is already removed from the DOM and thus hidden from the accessibility tree, so the `aria-hidden` attribute is not necessary.

As with using any ARIA attributes, you should always test with assistive technologies, and preferably having people with disabilities test your work, to make sure that it's easy to understand, even with these hidden elements.

You should also know that setting `aria-hidden` to `false` will not expose the element to assistive technologies if any of its parents has this attribute set to `true`.

The `aria-hidden` attribute is used to hide elements from people who use assistive technology, such as screen reader users.

While it can be helpful for hiding purely decorative elements and duplicated content, it should be used sparingly to avoid hindering accessibility.

In general, all content and functionality available on the page should also be available to people using assistive technology. The use case for `aria-hidden` is very narrow and should be limited primarily to making the experience cleaner for screen reader users by removing purely decorative or duplicate information. Do not use `aria-hidden` to hide content that you don’t think screen reader users would be interested in. Screen reader users deserve to have access to all information on the page.

By following these best practices and testing the user experience, you can create inclusive online experiences for everyone.

What is the `aria-expanded` attribute, and how does it work?

The `aria-expanded` attribute is used for accessibility purposes to indicate if a control is expanded or collapsed. It's used in conjunction with collapsible widgets like menus, accordions and other disclosure widgets that control the visibility of content. The `aria-expanded` attribute is set to `true` if the control is expanded, or `false` if it is collapsed.

The information provided by `aria-expanded` allows people using screen readers to understand the current state of the control (whether it is expanded or collapsed).

The `aria-expanded` attribute is applied to the interactive element that toggles the visibility of a collapsible widget. For example, if a button toggles an expandable menu, the `aria-expanded` attribute is placed on the button.

When the menu is expanded, the `aria-expanded` attribute should be set to `true` like in this example:

```
<button aria-expanded="true">Menu</button>
```

When the menu is collapsed, it should be set to `false` instead.

```
<button aria-expanded="false">Menu</button>
```

You must always have the `aria-expanded` attribute set to either `true` or `false` on the controlling element. For example, if a button toggles the visibility of a menu, the default value for `aria-expanded` should be based on the default visibility of the menu.

If the menu is expanded by default, `aria-expanded` should initially be set to `true`. If the menu is collapsed by default, `aria-expanded` should initially be set of `false`.

The value of `aria-expanded` should be updated dynamically using JavaScript as the user interacts with the element.

Additionally, the properties, `aria-controls` and `aria-owns` can be used in combination with `aria-expanded` to establish a programmatic connection between the

controlling element and the element it controls.

Let's start with `aria-controls`. When used with `aria-expanded`, the `aria-controls` attribute is used to specify the element being controlled. For example, a button might expand or collapse a list acting as a menu. The value of `aria-controls` will be the `id` of the controlled element (the menu list in this example).

```
<button aria-expanded="false" aria-controls="menu1">Menu</button><ul id="menu1"><li>...</li><li>...</li></ul>
```

Notice that the list immediately follows the controlling button. For expandable controls like this, it is best to have the expanded content immediately follow the element that controls it in the DOM. This prevents screen reader users from having to search for the expanded content, and makes it easier for keyboard users to navigate through any interactive controls in the expanded content.

If it is not possible to place the expanded content immediately after the controlling element, the `aria-owns` attribute allows you to virtually move it after the control in the accessibility tree. This allows assistive technology like screen readers to pretend the expanded content is placed directly after the control in the DOM.

```
<button aria-owns="menu1" aria-expanded="true">Menu</button><main>
<!-- an entire page worth of content ⟶
</main><ul id="menu1"><li>...</li><li>...</li></ul>
```

There are drawbacks to using the `aria-owns` attribute. It creates unnecessary verbosity for screen reader users since most screen readers will automatically read out the entire contents of the expanded element when first expanded. It also does not change the tab order, potentially forcing keyboard users to tab through all of the other interactive controls on the page before reaching the expanded content, unless you manage the tabbing order with JavaScript.

Ideally, the expandable content should be placed after the control element, and the `aria-owns` attribute should only be used in a worst case scenario when that is not possible. If it must be used, you will need to thoroughly test with a wide range of screen readers and browsers to ensure that your implementation is accessible for everyone.

And just a reminder, when you use either `aria-controls` or `aria-owns`, the value of `aria-expanded` must continue to be updated as the control is expanded and

collapsed.

The `aria-expanded` attribute indicates whether a control is expanded or collapsed. This information is essential for screen reader users, helping them understand the current state of collapsible elements like menus, accordions, and other similar disclosure widgets.

By using `aria-expanded` correctly, you can create an intuitive user experience for everyone.

What is the `aria-live` attribute and how does it work?

The `aria-live` attribute creates a live region on your page which can be used to notify screen reader users of dynamic content changes in the live region as they occur.

Common uses of live regions include messages that are displayed after an action has been taken (such as an error message or confirmation), content that updates periodically (such as a ticker, marquee, or countdown timer), or general status messages (such as updates about a process).

Because the reading focus for screen readers can only be at one place at a time, screen reader users will not notice a content change if their focus is on another part of the page. Live regions allow screen reader users to be automatically notified of changes that happen on the page in real time. Without a live region, screen reader users might miss important content updates available to sighted users, since a sighted user has the ability to scan the entire page.

There are three possible values for this attribute, based on the priority of the information.

If you set `aria-live` to the value `assertive` that means that the update is very important. It has the highest priority, so the user should be notified immediately.

This means that the screen reader will interrupt any announcement it is currently making to announce the content change in the live region. Such interruptions can be extremely disruptive, so the `assertive` value should only be used for time-sensitive or critical notifications.

```
<div aria-live="assertive"><p>Your session will expire in 30 seconds.</p>
</div>
```

The next value in order of priority is `polite`.

This value means that the update is not urgent, so the screen reader can wait until any current announcement is finished or until the user stops typing before announcing the update. Most live regions will use the `polite` value.

```
<div aria-live="polite"><p>File successfully uploaded</p></div>
```

The lowest priority value for `aria-live` is `off` which means that the update will not be announced unless the content is in an element that currently has keyboard focus. Realistically, this value is almost never used as the use case is very narrow and it is not implemented consistently (if at all) across screen readers. If you need live regions, plan on using `polite` for everything except critical messages that need `assertive`.

It's also important to note that the `aria-live` attribute is not required if the updated information is contained in an element with an ARIA role of `alert`, `log`, `marquee`, `status`, or `timer`, as these roles already have default `aria-live` values. But the default value can be changed by explicitly setting `aria-live` on the element.

Choosing the right `aria-live` value depends on the priority of the updated information.

If the updates are urgent, you should notify the user immediately with `assertive`. But you should use this only if the updates are really urgent because sudden interruptions can potentially disorient users and affect the user experience.

If the update can wait until the current task is finished, you should use `polite` instead.

The `aria-live` attribute lets assistive technologies know when the content is changing dynamically on a web page. This helps users with disabilities stay updated on important announcements and updates.

By using `aria-live` appropriately, you can ensure that users are aware of these updates based on their priority.

What are some common ARIA states used on custom control elements?

Semantic form control elements like `input`, `select`, `textarea`, `button`, and `fieldset` have built-in states that are conveyed to assistive technologies.

For example, you could use the `disabled` attribute to disable a button or the `checked` attribute to indicate that a checkbox is checked.

But if you are creating a custom control element, you need to use ARIA attributes to convey the state of the control to assistive technologies.

In this lecture video, we will discuss a few common ARIA states that you can use on custom control elements.

The first ARIA state we will discuss is `aria-selected`. This state is used to indicate that an element is selected. You can use this state on custom controls like a tabbed interface, a listbox, or a grid.

Here is an example of how you can use `aria-selected` on a custom tab control:

```
<div role="tablist"><button role="tab" aria-selected="true">Tab 1</button
><button role="tab" aria-selected="false">Tab 2</button><button role="ta
b" aria-selected="false">Tab 3</button></div>
```

Tabs are used to display multiple panels of content in a limited space. The `aria-selected` state is used to indicate which tab is currently selected.

When the user selects a tab, the `aria-selected` state of the selected tab is set to `true`, and the `aria-selected` state of the other tabs is set to `false`.

Another common ARIA state is `aria-disabled`. This state is used to indicate that an element is disabled only to people using assistive technologies, such as screen readers. It is important to note that `aria-disabled` does not actually disable the element. It is up to you, the developer, to make it look and act like a disabled element. This attribute is also commonly used on native HTML elements in place of the `disabled` attribute. Which one you choose will depend on the context the button is being used.

Here is an example of how you can use `aria-disabled` on a custom edit button:

```
<div role="button" tabindex="-1" aria-disabled="true">Edit</div>
```

The `aria-disabled` attribute is used to tell screen reader users that the edit button is disabled and cannot be interacted with. Again, it does not actually disable the button. When using `aria-disabled`,you will need to apply styling and JavaScript to make the control look and behave like a disabled button.

In most cases, you will probably use the native button element, but there are cases where you might need to use a custom control. So, it is essential to know

how to convey the state of the control to assistive technologies.

The next ARIA state we will discuss is `aria-haspopup`. This state is used to indicate that an interactive element will trigger a popup element when activated. You can only use the `aria-haspopup` attribute when the popup has one of the following roles: `menu`, `listbox`, `tree`, `grid`, or `dialog`. The value of `aria-haspopup` must be either one of these roles or `true`, which defaults to the `menu` role.

Here is an example of a file editor menu that uses `aria-haspopup`:

```
<button id="menubutton" aria-haspopup="menu" aria-controls="filemenu" aria-expanded="false">File</button><ul id="filemenu" role="menu" aria-labelledby="menubutton" hidden><li role="menuitem" tabindex="-1">Open</li><li role="menuitem" tabindex="-1">New</li><li role="menuitem" tabindex="-1">Save</li><li role="menuitem" tabindex="-1">Delete</li></ul>
```

The `aria-haspopup` state is used to indicate that the `File` menu button will open a popup menu when activated. Screen reader users may hear this additional information when they navigate to the button.

You will need to use JavaScript to show and hide the popup menu, and to implement proper keyboard support for interacting with the menu. Also, please note that the ARIA `menu` role refers to a very specific type of menu. It generally refers to a list of actions that the user can invoke, similar to a menu on a desktop application. It does not include more common uses of what we typically refer to as "menus", such as navigation menus. Realistically, most "menus" you create on the web will not be ARIA menus and you will not use `aria-haspopup` with them.

The next ARIA state we will discuss is `aria-required`. The `aria-required` attribute is used to indicate that a field needs to be filled out before the form is submitted. Here is an example of working with the `aria-required` attribute for a custom form control.

```
<div id="name-label">Full Name*</div><div role="textbox" contenteditable aria-labelledby="name-label" aria-required="true" id="name"></div>
```

We need to use the `contenteditable` attribute so users can type in their input. We are also using the `aria-required` attribute set to `true` to indicate that this custom form control is required.

To make the form control look like a normal form control, you would need to add CSS. You would also need to add JavaScript to prevent the form from being submitted without content.

If the label already has the word `required`, then you should omit the `aria-required` attribute. This ensures that screen readers only announce the word required once.

In most cases, you will probably use the native `label` and `form` elements with the `required` attribute. But if you need to create a custom form control, then it is important to add the `aria-required` attribute when necessary.

Additionally, the `aria-required` attribute can also be used on native form inputs, such as the `input`, `textarea`, and `select` elements. This is often preferred to the native required attribute, since the required attribute may have potential usability and accessibility concerns, particularly with the default error handling provided by the browser. Ultimately, you will need to test in order to determine which attribute is best for your situation.

The last ARIA state we will discuss is `aria-checked`. This attribute is used to indicate whether an element is in the checked state. It is most commonly used when creating custom checkboxes, radio buttons, switches, and listboxes.

Here is an example of how you can use `aria-checked` on a custom checkbox control:

```
<div role="checkbox" aria-checked="true" tabindex="0">Checkbox</div>
```

Native checkbox elements have a built-in `checked` state that is conveyed to assistive technologies. But if you are creating a custom checkbox control, you will need to use the `aria-checked` attribute to indicate its state.

When the user interacts with the custom checkbox control, you will need to use the `aria-checked` state to reflect the new state of the checkbox. When the checkbox is checked, the `aria-checked` attribute is set to `true`. When it is unchecked, it is set to `false`.

Native elements typically have better support and built-in accessibility features.

However, if you must create custom controls, using ARIA attributes is essential to convey the state of these controls to assistive technologies effectively.

As always, test your work to ensure that ARIA attributes are applied correctly and that the custom control functions in a way that is both accessible and user-

friendly.

What is the `aria-controls` attribute, and how does it work?

The `aria-controls` attribute is used to create a programmatic relationship between an element that controls the presence of another element on the page, and the element it controls. This relationship can help screen reader users better understand how the control works. Common uses include a set of tabs that control the visibility of different sections of content, or a button that toggles the visibility of a menu.

Let's take a look at an example to see how this works. In this example of a tabbed interface, we have a `div` element with a set of three buttons:

```
<div role="tablist"><button role="tab" id="tab1" aria-controls="section1" ar
ia-selected="true">
  Tab 1
 </button><button role="tab" id="tab2" aria-controls="section2" aria-sele
cted="false">
  Tab 2
 </button><button role="tab" id="tab3" aria-controls="section3" aria-sele
cted="false">
  Tab 3
 </button></div>
```

Each button represents a tab and has a `role` attribute set to `tab`. In most tabbed interfaces, the first tab panel is visible by default, so the first tab button has an `aria-selected` attribute set to `true` to indicate that its associated section of content is currently visible The `aria-controls` attribute is used to associate each button with the section of content that it controls.

Here are the three sections of content that correspond to the tabs:

```
<div id="section1" role="tabpanel" aria-labelledby="tab1">
 Section 1 content
</div><div id="section2" role="tabpanel" aria-labelledby="tab2" hidden>
 Section 2 content
</div><div id="section3" role="tabpanel" aria-labelledby="tab3" hidden>
```

```
  Section 3 content
</div>
```

Each section of content has a `role` attribute set to `tabpanel` and an `aria-labelledby` attribute that points to the corresponding tab to give the panel an accessible name. The `hidden` attribute is used to hide the sections of content that are not currently selected. Each section id matches the value of the `aria-controls` attribute on the corresponding button. The `section1` id matches the `aria-controls` attribute on the first button, `section2` matches the `aria-controls` attribute on the second button, and `section3` matches the `aria-controls` attribute on the third button. This is how the association between the tabs and their sections of content is established.

To switch between the different sections you will need to use JavaScript to update the `hidden` attribute on the section and the `aria-selected` attribute on the button based on which section is currently visible. Only one section can be visible at a time and it should not have the `hidden` attribute and `aria-selected` should be set to `true` on its button. The remaining hidden sections should all have the `hidden` attribute and `aria-selected` should be set to `false` on their buttons.

Tabs are a common use case for the `aria-controls` attribute, but it can be used in other scenarios where one element controls the visibility or behavior of another element. Other examples include accordions, dropdown menus, and modals.

The next time you design a user interface that involves controlling the visibility of elements, consider using the `aria-controls` attribute to establish the relationship between the controlling element and the controlled element

---

What is the `aria-describedby` attribute, and how does it work?

The `aria-describedby` attribute is used to provide additional information about an element to screen reader users by referencing existing content on the page. It creates a programmatic association between the element and the content (technically referred to as an accessible description), which screen readers can use to inform users of the additional information when they interact with the element.

The most common use for `aria-describedby` is to associate instructions and error messages with form inputs. Due to the various methods screen reader users

have for navigating a page, they may miss these messages when navigating between inputs. Using `aria-describedby` helps ensure that they will hear them.

Let's take a look at a few examples to understand how it works. In this first example, we have a `form` element that accepts a password:

```
<form><label for="password">Password:</label><input type="password" id="password" aria-describedby="password-help" /><p id="password-help">Your password must be at least 8 characters long.</p></form>
```

We are using a `label` element for the `Password` text and associating that with the password `input` field.

We also have a paragraph element that describes the password requirements. We are using the `aria-describedby` attribute to associate the password `input` field with the password requirements in the paragraph element. When a screen reader user interacts with this `input`, their screen reader will announce the name of the `input` (`Password`) and may then also announce the password requirements. This is not an absolute guarantee however as some screen readers may not automatically announce the additional content, or may only announce it in specific circumstances. This is not uncommon. Each screen reader is different and handles ARIA attributes in their own way. This does not negate the use of `aria-describedby`, as it will benefit screen reader users in general.

Another good use case for the `aria-describedby` attribute is when you have a delete `button`. Here is an example of a delete `button` followed by a message describing what will happen when the button is clicked:

```
<button aria-describedby="delete-message">Delete</button><p id="delete-message">Warning! All deletions are permanent.</p>
```

Just like in the earlier example, we associate the delete button with the message using the `aria-describedby` attribute. The `id` value and the value of the `aria-describedby` attribute must match.

The `aria-describedby` attribute is a powerful attribute that can be used to help ensure that additional information about an element is provided to screen reader users when they interact with the element. It is most commonly used to associate instructions and error messages with form inputs in order to reduce the chance that screen reader users will miss these messages as they navigate the form

Working with Accessible Media Elements

Working with accessible media elements means ensuring that audio and video content is usable by everyone, including individuals with visual or hearing impairments. For images, the crucial step is providing concise and descriptive `alt` text so screen readers can convey their meaning. When embedding audio or video, always include the `controls` attribute to give users playback options. More importantly, provide alternatives like closed captions for the deaf and hard-of-hearing, and transcripts for audio-only content. For video that conveys significant visual information without spoken dialogue, an audio description track or a detailed descriptive transcript is essential to ensure blind or low-vision users can understand the visual context.

When is the `alt` attribute needed, and what are some examples of good `alt` text?

Alternative text, often abbreviated as `alt` text, is a brief text description of an image. It provides essential information about the image for users who cannot see it, such as people who use screen readers and other assistive technologies.

It's essential for making websites accessible to people with visual disabilities.

Alternative text is also used by search engines to understand images. Some browsers may also display it when an image is not loaded properly. This can happen when the image file is missing or when the user has connectivity issues. So it's helpful for many different purposes.

A person with a visual disability will not be able to know what's in the image unless it's described in the alt text. An example of bad alt text for an image of a puppy would be "A cute puppy." This text is not specific enough to convey the important details of the image, like what does the puppy look like? Where is the puppy? Are there any important objects around the puppy?

Let's improve this. An example of good alternative text would be: "A black and white puppy with an orange collar lies on its belly in the sand, looking off to the side. A bright orange ball rests near its front paws."

Here you can see this in HTML with an image element, the `alt` attribute, and a more detailed description:

```
<img src="puppy.png" alt="A black and white puppy with an orange collar l
ies on its belly in the sand, looking off to the side. A bright orange ball rests
near its front paws." />
```

It is important to note that there is no one correct way to write alt text for an image. What you include in your description will depend on the context the picture is used in. For example, if the puppy picture was on a website about dog breeds, you might want to include more specifics about the physical description of the puppy and possibly ignore the fact that the puppy is at the beach playing with an orange ball. Ultimately, the `alt` text you use for an image should reflect the primary purpose for including the image on the page, and the information you provide should give people who can't see the image the necessary details to understand that purpose.

Here we have another example of a beautiful tropical resort. Let's describe it.

An example of bad `alt` text for this image would be "Resort."

This is too short and it doesn't provide enough information about the image. To improve this description, you could expand it to include the most important elements of the image:

"Tropical resort featuring a swimming pool surrounded by palm trees and bungalows."

You can use the alt attribute in HTML like this:

```
<img src="resort.png" alt="Tropical resort featuring a swimming pool surro
unded by palm trees and bungalows." />
```

Now that you know what makes good and bad `alt` text, let's see some of the best practices.

- You should try to keep `alt` text short. It should be detailed enough to understand the image but not so long that it becomes confusing.
- You should not try to describe every little detail. Focus on the most important aspects of the image.
- Generally, you don't need to start with "image of" or "picture of." You can just start the description directly.

- Also, if there's similar text around the image, you don't need to write it again.
- It's usually recommended to end the alt text with a period for consistency.
- If the image is a link to another page, instead of describing the image itself, the `alt` text should describe what will happen if users click on it.

For example, if your website has a right arrow icon that takes the user to the next page, instead of writing an alt text that only says "right arrow", like in this example, where you can see the alt attribute with this description:

```
<a href="about.html"><img src="arrow-right.png" alt="Right arrow." /></a
>
```

You should write something like this instead, where the `alt` text describes what will happen if users clicks on the image. They will go to the next page.

```
<a href="about.html"><img src="arrow-right.png" alt="Go to next page." /
></a>
```

Only images that convey important information should have `alt` text. If an image is only used for decorative purposes, it should have `null` (empty) `alt` text, so it can be ignored by screen readers and other assistive technologies.

Here is an example of an empty `alt` attribute is empty:

```
<img src="decorative_image.jpg" alt="" />
```

Every image on your website should have an `alt` attribute, even if it's empty. If you omit the `alt` attribute completely, some screen readers will read the file name instead, which can be distracting for people who use assistive technologies, so this is not recommended.

Finally, before your website is live, you should carefully test if screen readers can read the `alt` text correctly.

Writing effective `alt` text is essential for creating accessible web content. As a web developer, by providing clear descriptions of your images, you can make sure that everyone can engage with your website in an inclusive online experience.

What are the accessibility benefits for good link text, and what are examples of good link text?

Let's take a look at the benefits of writing good link texts in the context of accessibility, and some examples of good link texts.

The first visible benefit of a good link text is that it makes it easier for everyone to find information quickly. Descriptive links help users know where they're headed and what they'll access. This ensures the user doesn't feel lost and improves the overall user experience.

For those using screen readers, a clear and descriptive link text is a must. Screen readers read the link text aloud, so a text like "Read our accessibility guide" is way better than "Click here."

Making link text clear and descriptive isn't just beneficial for those living with visual impairments. Descriptive links also help people with cognitive disabilities by providing clear context.

Here are some best practices to keep in mind while writing link texts:

- Make sure links are visually distinct by using underlining and other visual cues, so users can easily identify and navigate them.
- Avoid generic link texts like "here", "click here", and "more-info" as they don't provide any useful information.
- Aim for concise and descriptive link texts, ideally between 2-5 words, that convey the link's purpose.
- Avoid jargon and abbreviations that users may not understand.
- Focus on the destination, not the action. For example, "user behavior results", instead of "click here to read more".
- Don’t repeat the same link text for different destinations.
- Place links in a way that they make sense within the surrounding text. For example, "for more details, visit our events page" instead of "Click here for more".

Here are some examples of good link texts for specific use cases, compared to less helpful ones. Let's say you want to link to a page providing details about an event like this:

```
<a href="webinar-details-link">Details</a>
```

`Details` is vague and doesn’t provide specific information about what the user will find if they click the link. Without additional context, users might not know if the link leads to details about a webinar, a product, a policy, or something else.

Now, here is an example of good link text:

```
<a href="webinar-details-link">
  Get details about our upcoming webinar
</a>
```

This link text gives users context about the content they will find, making it easier to decide whether they want to click on it. It reduces ambiguity by specifying that the link is related to a webinar.

Here is an another example linking to a post in a blog:

```
<a href="/blog-post-link">Read more</a>
```

The link text `Read more` is not ideal in terms of accessibility because it lacks context.

Here is a better link text example:

```
<a href="/blog-post-link">
   Read our latest blog post on web accessibility
</a>
```

This updated link text provides users with a clearer idea of what to expect and why they might want to click the link, which is particularly important for those using screen readers.

Let's take a look at one more example. Let's say you want to provide more information about a certain topic:

```
<a href="/link-to-topic">More info</a>
```

`More info` is also vague and can be problematic for accessibility. It doesn't provide specific details about the link's destination, which can be confusing for

users relying on screen readers or those who might be navigating the site in a non-linear way.

Here is a better example for link text:

```
<a href="/link-to-topic">
  Learn more about our accessibility efforts
</a>
```

This updated link text is better because it clearly describes what the user will find if they click the link. Users can immediately understand that the link leads to information about your accessibility efforts.

What are good ways to make audio and video content accessible?

Multimedia, especially video, has become the go-to format for sharing information online.

With this surge in content, there's a growing need to ensure everyone can access and enjoy video and audio content online, irrespective of their abilities or environment.

Making your audio and video content accessible isn't just a nice-to-have, it's essential for reaching a wider audience. Let's look at some inexpensive ways to make your video and audio content more accessible.

A video is not just about visuals but also audio, so the first thing you should consider doing is adding captions or subtitles to your video content.

Captions provide the text version of spoken words and important non-verbal sounds, like music or laughter, synchronized with the video.

Subtitles on the other hand are essential for people who don't understand the language you're speaking. This helps not only people who are deaf or hard of hearing but also those watching videos in noisy or quiet environments.

To add captions or subtitles to your video or audio content, you can use the `track` element inside your `video` or `audio` element:

```
<!-- Video -->
<video controls src="video.mp4"><track
  src="captions.vtt"kind="captions"srclang="en"label="English"/></video
```

```
><!-- Audio -->
<audio controls src="audio.mp3"><track
  src="captions.vtt"kind="captions"srclang="en"label="English"/></audio
>
```

The `kind` attribute is used to tell the track element how it should be used. Valid values for the `kind` attribute include `captions`, `subtitles`, `chapters`, and `metadata`.

The `srclang` attribute represents the language for the `track` content.
The `label` attribute is a descriptive title for the text track that browsers use to identify and display it in the list of available text tracks.

Another important thing to consider is providing a transcript for your audio and video content. A transcript is a text version of all the spoken words in your audio or video. Unlike captions, transcripts don't need to be synchronized with the media. Transcripts are useful for deaf people and those hard of hearing. They're also beneficial for people who prefer reading instead of watching or listening. Transcripts also make your content searchable, allowing users to quickly find specific parts of your audio or video. If you have a video or audio on a website, you can simply add the transcript below the audio or video:

```
<audio controls><source src="audio.mp3" />
  Your browser does not support the audio element.
</audio><!-- Transcript -->
<h3>Transcript</h3><p>
  [Speaker 1]: Welcome to the tutorial on making accessible content
</p><p>
  [Speaker 2]: Today, we'll cover captions, transcripts, and more.
</p><!-- Rest of transcript -->
```

If you're publishing videos on a video-sharing platform like YouTube or Vimeo, they have automatic captions and transcripts for videos. But if you're not satisfied, you can use services like veed.io, Rev, Amara, and Descript.

Other ways to make your video and audio content accessible include:

- Adding a sign language overlay for videos for deaf people and those hard of hearing.
- Providing volume and speed controls.
- Ensuring good contrast for on-screen text.

- Offering multiple formats.

What are some ways to make web applications keyboard accessible?

Many users rely on keyboards instead of mice due to physical disabilities, repetitive strain injuries, or personal preference. This includes users of screen readers and those who may not have a working mouse. Keyboard accessibility ensures these users can navigate web applications effectively without barriers.

Let's look at some practical techniques you can employ to make web applications keyboard accessible.

Many users use the `Tab` key to navigate through interactive elements on a webpage. The `tabindex` attribute is used to make elements focusable and define the relative order in which they should be navigated using the keyboard. It takes a numerical value, which can be positive, zero, or negative.

Here is the basic syntax:

```
<element tabindex="number">Element Text</element>
```

It is important to never use the `tabindex` attribute with a value greater than `0`. Instead, you should either use a value of `0` or `-1`.

You can set the `tabindex` to `-1` to allow it to receive programmatic focus. This is used in a variety of situations for "focus management", such as when you need to set the focus on a heading, error message, or dialog, or you are building custom components such as tabbing interfaces or tree components.

```
<p tabindex="-1">Sorry, there was an error with your submission.</p>
```

Setting `tabindex` to `0` allows you to add an element that does not receive keyboard focus by default into the page's natural tab order. This allows keyboard users to `Tab` to the element. This is primarily used when building custom elements that need keyboard functionality.

```
<div role="combobox" tabindex="0">
```

`accesskey` is another attribute you can use to make your web project keyboard accessible. It allows you to define a key that focuses on or activates a

particular element:

```
<button accesskey="s">Save</button><button accesskey="c">Cancel</b
utton><a href="index.html" accesskey="h">Home</a>
```

In the above code:

- `accesskey="s"` assigns the key `S` to the `Save` button. On most browsers, pressing `ALT + S` (on Windows) and `CTRL + Option + S` (on Mac) will activate this button.
- `accesskey="c"` sets the key `C` to the `Cancel` button, allowing users to activate it using `ALT + C` (Windows) and `CTRL + Option + C` (Mac).
- `accesskey="h"` assigns the key `H` to the `Home` link, allowing users to navigate to the homepage using `ALT + H` (Windows) and `CTRL + Option + H` (Mac).

Please note that the exact key combination to activate the accesskey might vary depending on the browser and operating system. It's typically `ALT + Specified Key` on Windows and `CTRL + Option + Specified Key` on Mac.

Another way to make the keyboard accessible in your apps is to make sure you provide clear focus indicators. If you feel the default browser focus indicator is not enough, you can override it by targeting the focus state of the element.

Here is an example of styling the focus state for an HTML element:

```
element:focus {
  outline: 2px solid #005fcc;
}
```

The `outline` property is used to define the outline around the element. This example sets the outline to a solid blue line with 2 pixels set for the thickness. The focus indicator should be styled in a way that makes it obvious which element currently has focus. In order to be accessible, the indicator must have a minimum color contrast of at least 3:1 with the background color it covers.

You should also avoid keyboard traps, which occur when a user cannot move focus away from a certain element in components like modals and popups